



FOR JAVA DEVELOPERS MOVING TO NESTJS

N+1. Or nothing.

Spring Boot to NestJS, episode 14: TypeORM migrations, relations and queries

Same relation, opposite default. A generated migration is a diff: read it.

JPA can fetch a lazy collection when you touch it: 21 queries for 20 orders. This TypeORM mapping leaves it unloaded unless you ask: 1 query, and 0 lines counted.

Paging a one-to-many must page the parents: a plain LIMIT on the join returned 5 rows from 3 orders.

A generated migration is a schema diff. Run on real rows, the type change failed safely on a NOT NULL column and nulled every value on a nullable one.

In scope

- @OneToMany and @ManyToOne on both stacks
- Loading relations: lazy against explicit
- Paging a one-to-many
- Generated migrations, run on rows

Not in scope

- Transactions: next episode
- TypeORM lazy relations with Promise types

One relation shape, one machine. The counts show behaviour, not constants.

01 The measurements

LOOP OVER order.lines, NOT FETCHED	Spring 21 queries, 60 lines; TypeORM 1 query, 0 lines
FETCH UP FRONT	join fetch / relations option: 1 query, 60 lines each
PLAIN LIMIT 5 OFFSET 5 ON THE JOIN	5 rows from 3 orders
TypeORM PAGE 2 OF 5 WITH LINES	2 queries, ORD-6 to ORD-10, 15 lines
GENERATED, RENAME	RENAME COLUMN, values kept
GENERATED, TYPE CHANGE, NOT NULL	DROP + ADD NOT NULL: refused, rolled back
GENERATED, TYPE CHANGE, NULLABLE	DROP + ADD: ran, every value null
HAND WRITTEN USING CAST	refused "abc", kept everything

20 orders, 3 lines each, the same schema on both stacks.

02 Ask for the relation

With this TypeORM mapping a relation is undefined unless requested. Request it, and for this one-to-many it arrives in one query.

probes.ts

```
@OneToMany(() => OrderLine, (line) => line.order)
lines!: OrderLine[];

const withLines = await orders.find({ relations: { lines: true }, order: { id: 'ASC' } });
```

the Spring side

```
@Query("select distinct o from Ep13Order o join fetch o.lines order by o.id")
List<Ep13Order> findAllWithLines();
```

Lazy collection loop: 21 queries; join fetch: 1 query; TypeORM without relations: order.lines undefined

Run on a real machine, 2026-09-30, by scripts/verify-relations.sh in the course repository.

03 Read a generated migration before it runs

The rename was right. The type change dropped and re-added the column: on a NOT NULL column Postgres refused it; on a nullable column it ran and every value became null. A hand-written USING cast refused bad data instead.

the hand-written alternative

```
ALTER TABLE "ep13_stock" ALTER COLUMN "quantity" TYPE integer USING "quantity"::integer
```

nullable column, generated DROP COLUMN then ADD integer: rows after [null, null] from ["123", "abc"]

Run on a real machine, 2026-09-30, by scripts/verify-relations.sh in the course repository.



Before you ship relations and migrations in TypeORM

1. Request relations explicitly where you read them
2. Do not trust a count computed from an unloaded relation
3. Page the parent rows, not the joined rows
4. Read every generated migration before running it
5. Write type changes by hand, with a USING cast
6. Run migrations against a copy of real data first

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-relations.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	TypeORM https://typeorm.io

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.